

CHAPTER FIVE

INTERRUPT OPERATION

BY

Suroj Burlakoti

Lecturer

DoECE, National College of Engineering

Chapter 5: Interrupt Operation

5.1 Polling versus Interrupt

5.2 Interrupt Processing Sequence

5.3 Interrupt Service Routine

5.4 Interrupt Processing in 8085

5.4.1 Interrupt Pins and Priorities

5.4.2 Using Programmable Interrupt Controllers (PIC)

5.4.3 Interrupt Instructions

5.5 Interrupt Processing in 8086

5.5.1 Interrupt Pins

5.5.2 Interrupt Vector Table and its Organization

5.5.3 Software and Hardware Interrupts

5.5.4 Interrupt Priorities

Interrupt

- Interrupt is signals send by an external device to the processor, to request the processor to perform a particular task or work.
- Mainly in the microprocessor based system the interrupts are used for data transfer between the peripheral and the microprocessor
- **Need for Interrupt:** Interrupts are particularly useful when interfacing I/O devices that provide or require data at relatively low data transfer rate.

Interrupt

- Types of Interrupt
 - External Interrupts:
 - Initiated via microprocessor pins by external devices.
 - Maskable Interrupt:
 - Can be enabled or disabled using EI and DI
 - Can be ignored by processor by making disable
 - Non-Maskable Interrupt
 - Has higher priority than Maskable interrupt
 - Can not be enable or disable by instruction
 - Internal Interrupts
 - Are activated internally by exceptional conditions such as overflow, division by zero and execution of an illegal opcode
 - Software Interrupt
 - Usually used to call operating system
 - Calls group of instruction (inbuilt subroutines) while interrupt happens

Polling versus Interrupt

Interrupt	Polling
1. Device notify CPU that it needs CPU attention	1. CPU constantly checks device status whether it needs CPU's attentions or not
2. An interrupt is generally hardware mechanisms	2. Polling is a protocol
3. Interrupt handler services a device	3. CPU services a device
4. Interrupt request line indicates that device needs servicing	4. Command Ready bit indicates that the device needs servicing
5. CPU is disturbed only when a device needs servicing, which saves CPU cycle	5. CPU has to wait and check whether a device needs servicing, which wastes lots of CPU cycles
6. An interrupt can be occurred at anytime	6. CPU polls device at regular interval
7. Efficient for when we need interrupt rarely	7. Efficient if we need repeatedly

Advantage of Interrupt Over Polling

- Fastest response to an event
- Save power consumption
- Efficient way to code

Learn With Suroj Burlakoti

Interrupt Structure

- A processor is usually provided with one or more interrupt pins on the chip.
- Therefore a special mechanism is necessary to handle interrupts from several devices that share one of these interrupt lines.
- There are mainly two ways of servicing multiple interrupts which are **polled interrupts** and **daisy chain (vectored) interrupts**.

Polled Interrupt

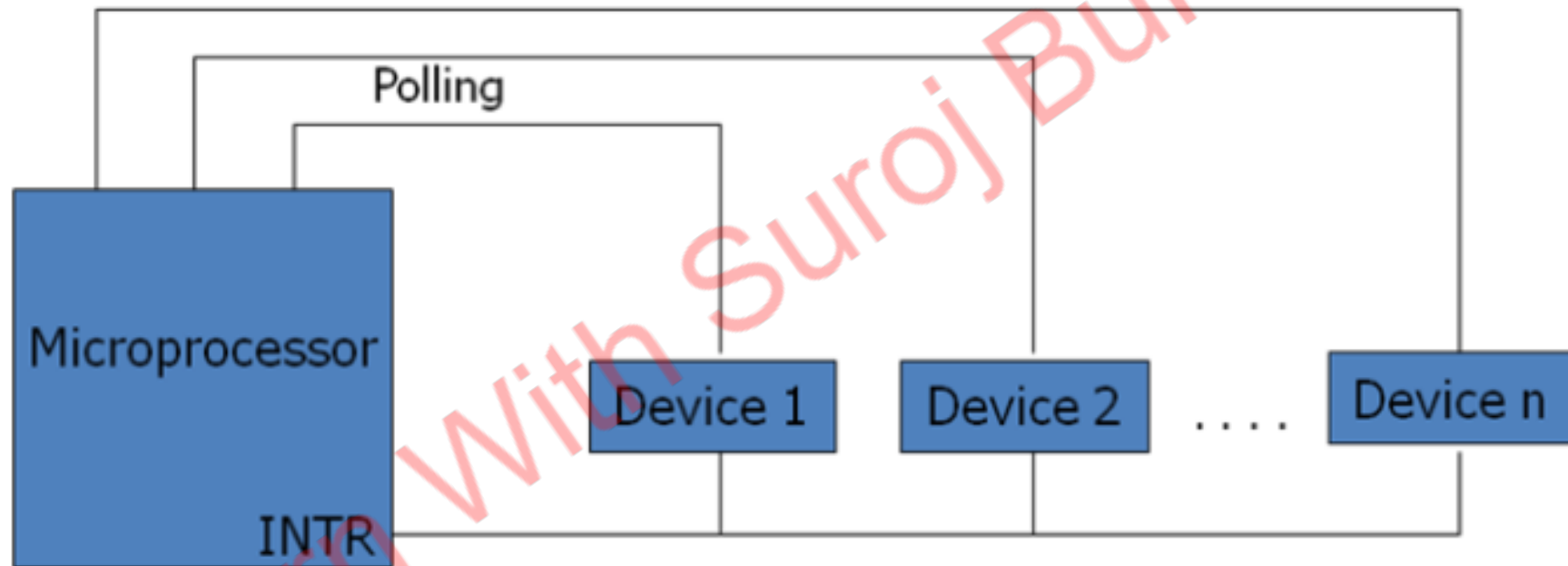


Fig: Polled Interrupt

Daisy chain (vectored) interrupt

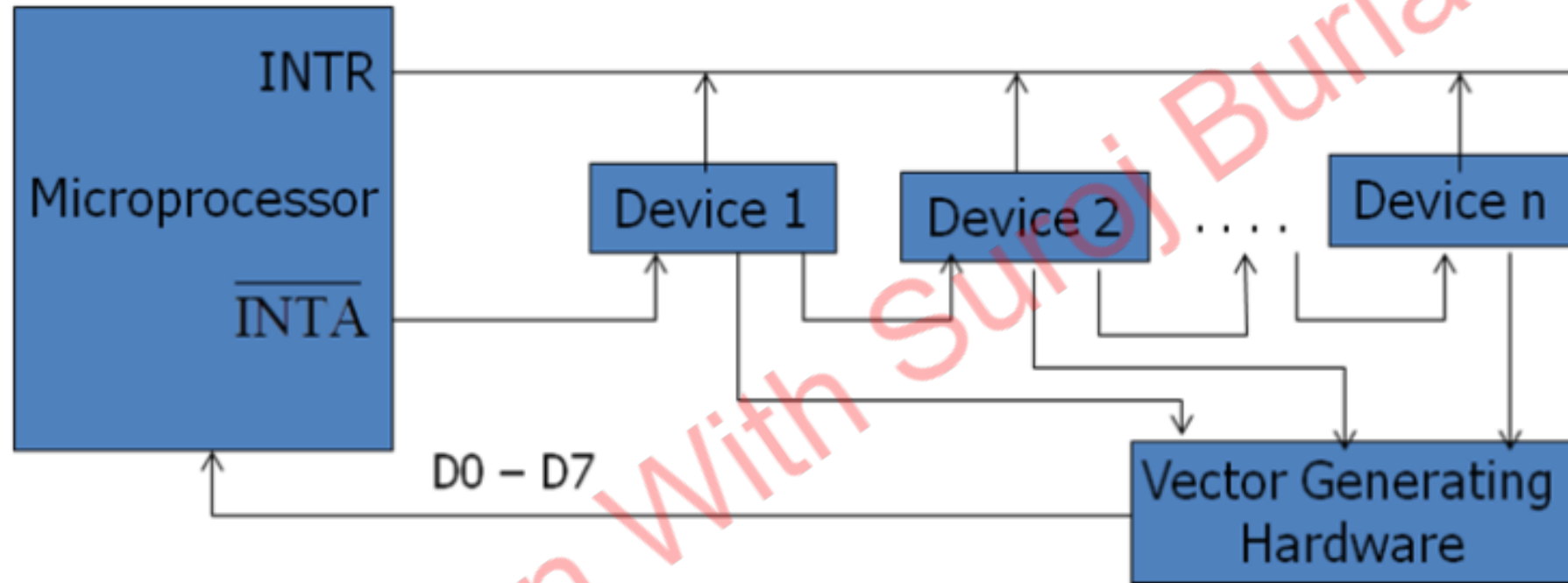
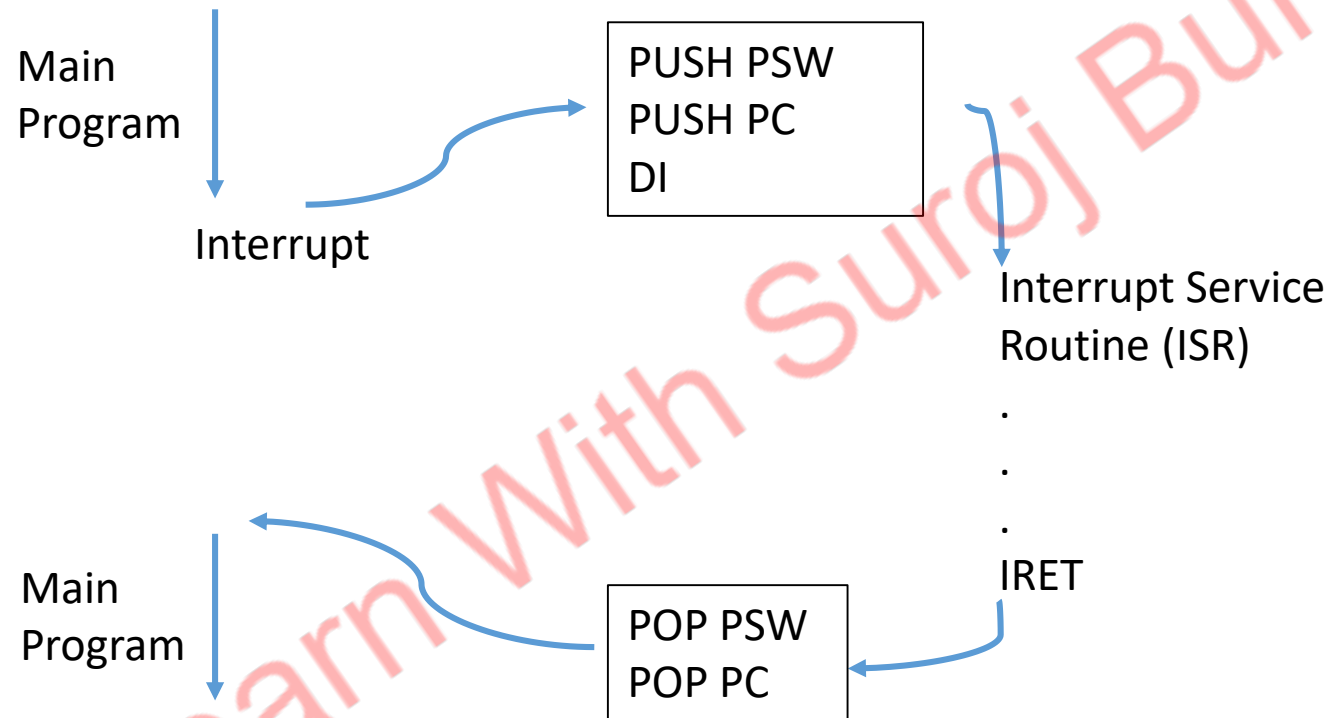


Fig: Vectored (Daisy Chain) Interrupt

Interrupt Processing Sequence

- The processor should enable the interrupt processor (EI)
- If INTR is HIGH, i.e. the I/O device issues an interrupt signal to the processor, the microprocessor completes the execution of current instruction, disables the interrupt (DI) and sends acknowledgement (*INTA*) signals to the device
- The processor begins to transfer its control to the requested routine from the device . This routine is called Interrupt Service Routine (ISR) and it resides at a fixed memory location.
- The ISR may be predefined or user defined to handle the interrupt processor
- For this, processor saves the information needed to resume the current program i.e. status of the program (PSW) and location of next instruction to be executed (PC) onto the stack
- The interrupt service routine (ISR) process the interrupt and at the end, the saved information is retrieved from the stack and normal processing resumes.

Interrupt Processing Sequence



Interrupt Service Routine (ISR)

- ISR is software routine that hardware invokes in response to an interrupt
- It examine an interrupt and determine how to handle it
- Main purpose is to process the interrupt and then return control to main program
- It must perform very fast to avoid delay in operation of device
- ISR is mainly responsible for following tasks:
 - Saving the processor context
 - Acknowledging the interrupt
 - Restoring the processor context

Interrupt processing in 8085

- Interrupt Pins and Priorities

Interrupt	Call location	Priority
TRAP	0024H	1 st
RST 5.5	002CH	4 nd
RST 6.5	0034H	3 rd
RST 7.5	003CH	2 th
INTR	-	5 th

Interrupt Pins and Priorities

1. TRAP

- Highest priority, non-Maskable interrupt
- Level and edge sensitive (i/p should go LOW to HIGH and stay HIGH to be acknowledged)
- Upon triggering this interrupt, the program control gets transferred to 0024H
- Used for critical events such as power failure

2. RST 7.5, 6.5, 5.5

- RST 7.5 is edge sensitive and RST 6.5, 5.5 are level sensitive
- Maskable interrupts
- Can be enabled under program control with instruction EI (Enable Interrupt) and SIM(Set Interrupt Mask)
- Priority lower than TRAP, in decreasing order RST 7.5, RST 6.5, RST 5.5

Interrupt Pins and Priorities

3. INTR

- Maskable interrupt
- Least priority
- EI and DI instructions used to set and reset the interrupt enable flip flop and thus enable and disable the interrupt process respectively

4. $\overline{INTA}$

- Acknowledgement of interrupt request
- Active low
- Interrupt enable flip flop is disabled
- Used to insert restart (RST) instruction

Interrupt Pins and Priorities

5. Software Interrupts

- Eight software interrupt in 8085, they are RST 0 to RST 7
- Software interrupts are program whose instructions are inserted at desired locations in a program
- The vector address can be calculated as
interrupt no. * 8 = vector address
e.g. RST 5 → $5 * 8 = 40 = 28H$ → Vector address = 0028H

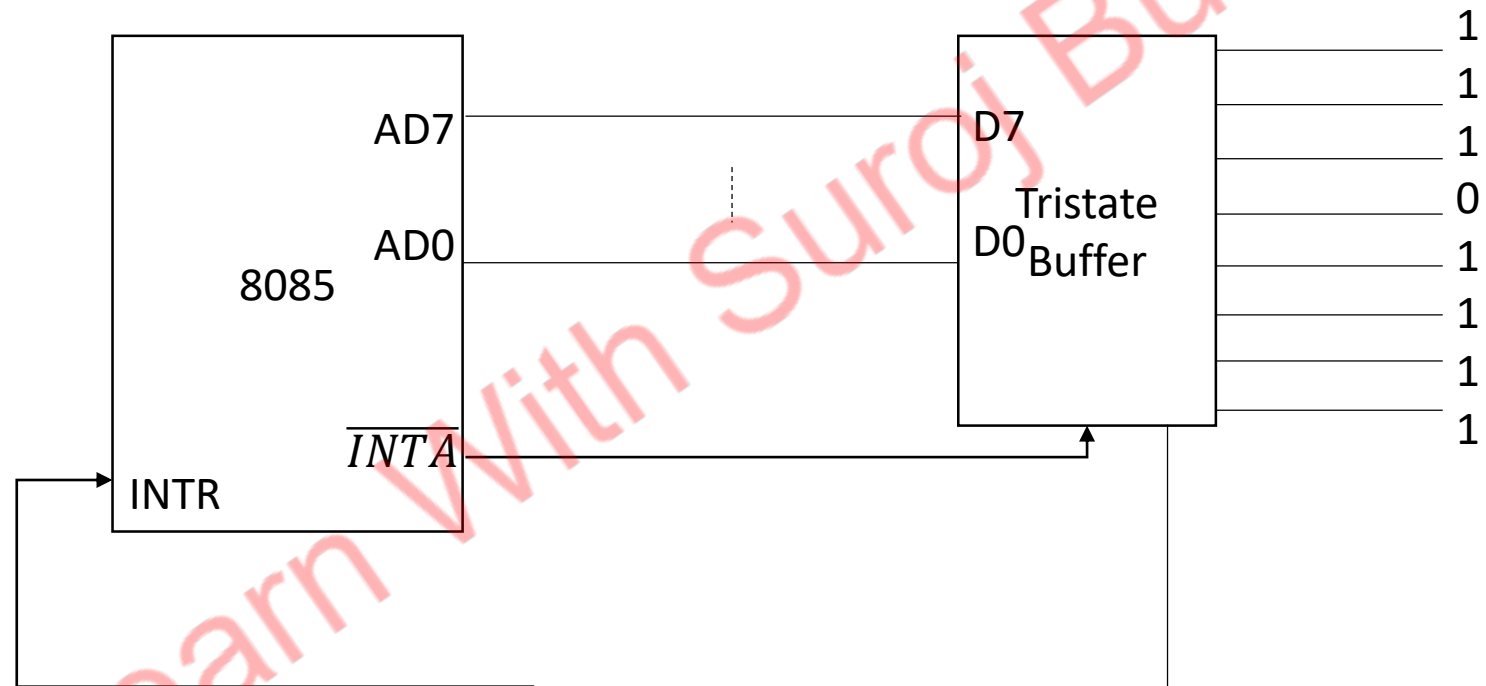
Interrupt	Vector Address
RST 0	0000H
RST 1	0008H
RST 2	0010H
RST 3	0018H
RST 4	0020H
RST 5	0028H
RST 6	0030H
RST 7	0038H

Interrupt Processing in 8085

- The interrupt is enabled by using EI instruction in main program
- If INTR is HIGH and EI is set then the microprocessor completes the current instructions disables interrupt enable flip flop and sends the signal called $\overline{INTA}$
- $\overline{INTA}$ signal is used to insert a RST or CALL instructions through external hardware
- RST transfers the program control to a specific memory location (predefined) and restart the execution at the memory location
- For this process it saves the memory address of next instruction on stack and program is transferred to the location specified by RST

Interrupt Processing in 8085

For RST 5
Address = EFH



Interrupt Processing in 8085

- In response to INTR high signal, 8085 sends $\overline{INTA}$ signal to enable the buffer and RST instruction is placed from external device to the data bus.
- Then the program control is transferred to memory location of respective RST instruction
- Jump instruction is written at that memory location to specify the address of the subroutine of service
- The interrupt is handled by the subroutine
- At the end of the subroutine, the return instruction transfer the program back to the point from where it was interrupted
- For this it enables EI and retrieves the stored address from the stack

Multiple Interrupts in 8085

Priority Interrupt Controllers (PIC)

- Normally in 8085, there is only one pin to receive interrupt from external device, which is not suitable for communicate with multiple peripherals
- So an external device called Priority Interrupt Controller (PIC) is used to overcome this limitation of 8085 and further extend the capability of 8085 interrupts
- The normally used PIC is 8259 PIC which acts as interrupt funneling device
- It is programmable device s and can be initiated to supply different RST or CALL instructions to the processor when interrupts arrives in any of its interrupt input lines.
- PIC also handles priority in its interrupt lines such that when two or more interrupts arrives at Interrupt requests pins simultaneously then the line with higher priority is processed

Multiple Interrupts in 8085

Priority Interrupt Controllers (PIC)

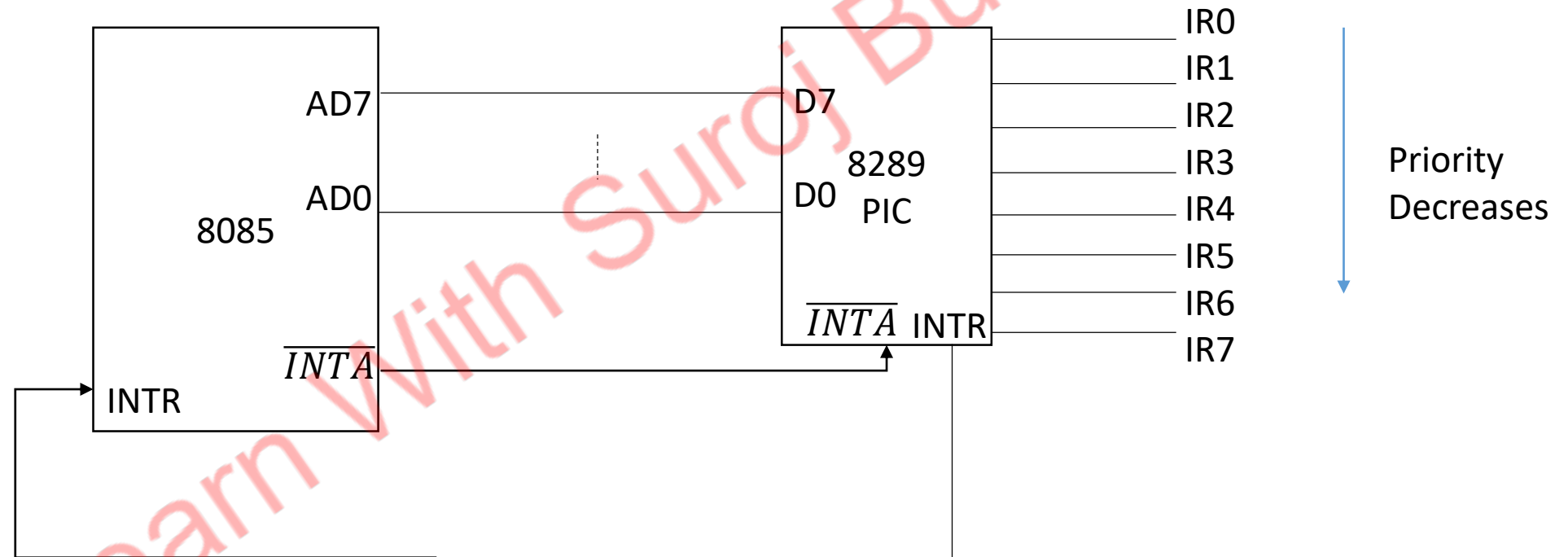


Fig: Multiple Interrupt with 8085 using PIC

Multiple Interrupts in 8085

Priority Interrupt Controllers (PIC)

Insert CALL instruction through PIC

- One or more interrupt request lines go HIGH requesting the interrupt
- The PIC resolves the priority and sends interrupt signal (INTR) to MPU
- MPU acknowledges the interrupt by sending $\overline{INTA}$
- After $\overline{INTA}$ has been received, the opcode of CALL instruction (CDH) is placed onto the data bus
- After call instruction, MPU sends more $\overline{INTA}$ signals after which PIC sends lower and higher address of the interrupt vector one by one
- The program sequence of the MPU is then transferred to the memory location specified by the call instructions

Interrupt Instructions in 8085

1. EI (Enable Interrupt)

- One byte instruction
- Sets interrupt enable flip-flop and enables the interrupt process

2. DI (Disable Interrupt)

- One byte instruction
- Resets interrupt enable flip-flop and disables the interrupt process

Interrupt Instructions in 8085

3. SIM (Set Interrupt Mask)

- One byte instruction
- Accumulator bit pattern before SIM instruction:

D7	D6	D5	D4	D3	D2	D1	D0
SOD	SOE	X	R7.5	MSE	M7.5	M6.5	M5.5

Serial
Output
Data

SOD
Enable

Reset
RST 7.5

Mask
Set
Enable

RST 7.5

RST 6.5

RST 5.5

Masking Bit → 0 = Available
1 = Masked

Interrupt Instructions in 8085

4. RIM (Read Interrupt Mask)

- One byte instruction
- Loaded to Accumulator

D7	D6	D5	D4	D3	D2	D1	D0
SID	I7.5	I6.5	I5.5	IE	M7.5	M6.5	M5.5

Serial
Input
Data

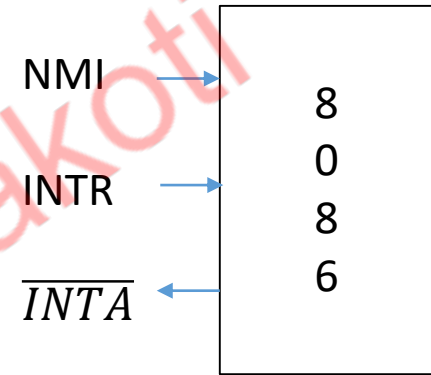
Pending Interrupt
1 = Pending

Interrupt
Enable Flag
1 = Enable
0 = Disable

RST 7.5 RST 6.5 RST 5.5
Interrupt Mask → 0 = Unmask
1 = Masked

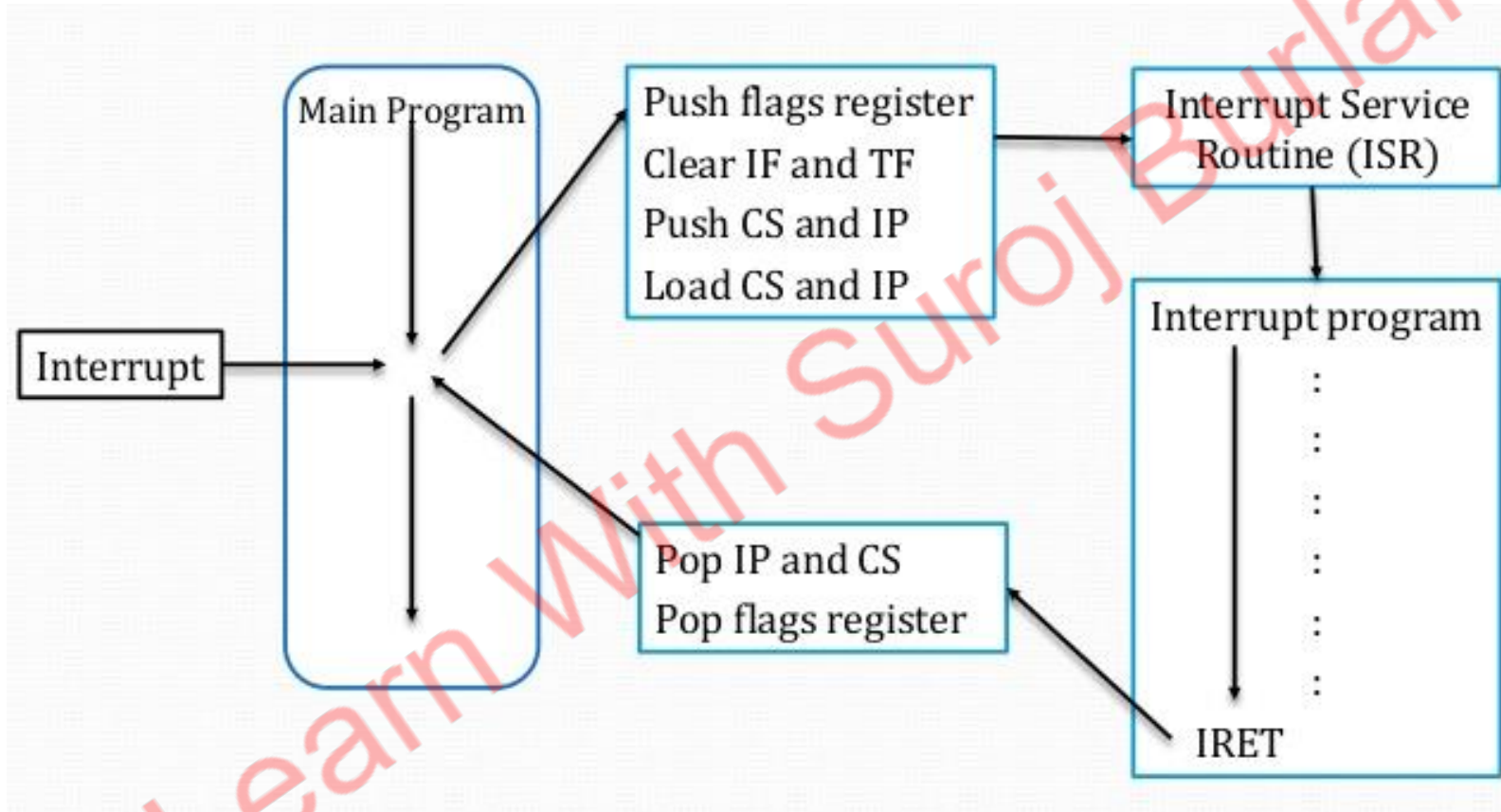
Learn With Suroj Burlakoti

Interrupt Processing in 8086

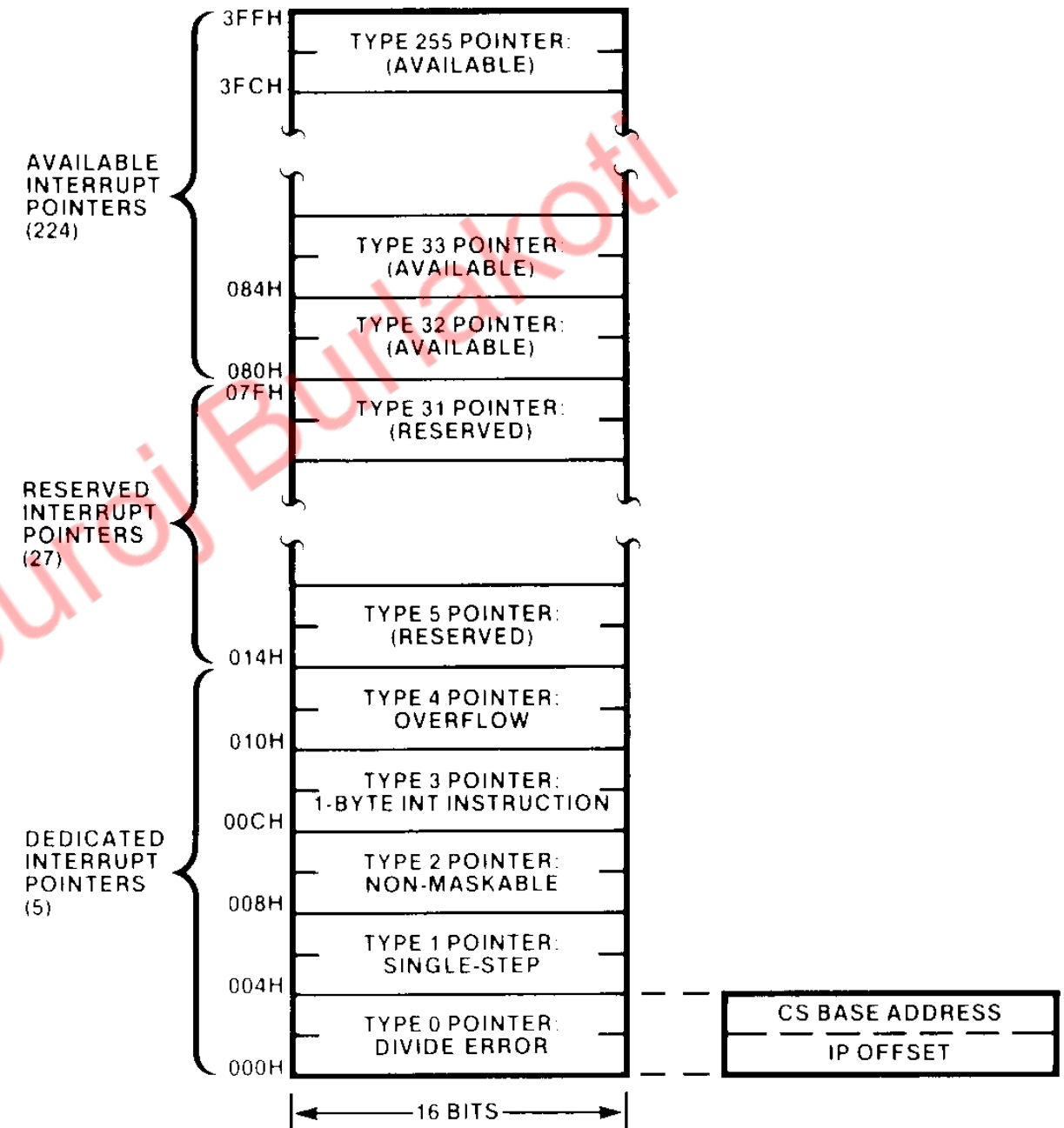


- Two interrupt pins
 - Non Maskable Interrupt (NMI)
 - Maskable Interrupt Request (INTR)
- A signal applied to any of these interrupt pins cause 8086 to transfer the program control to an interrupt procedure by stopping the execution of current program
- $\overline{INTA}$ acknowledge to the interrupt request and used to insert the appropriate type vector to the processor
- STI → Interrupt is enabled by STI (Set Interrupt) instruction
- CLI → Interrupt is disabled by CLI (Clear Interrupt) instruction

Interrupt Processing in 8086



Interrupt Vector Table and It's Organization



Interrupt Vector Table (IVT) and It's Organization

- The interrupt addresses of interrupt service procedure is called interrupt vector
- So an Interrupt Pointer Table (IPT) or Interrupt Vector Table (IVT) is the table containing 256 vectors, i.e. 4 byte address in the form of offset : segment (IP:CS) which begins in low memory at location 0
- The table provides for 256 interrupts, each with related 4 byte offset segment address. So it occupies from 0000H to 03FFH (1024 bytes)
- Each address in the table relates to BIOS or DOS routine for specific interrupt type
- First 5 interrupt vectors or pointers are dedicated for predefined function, followed by 27 interrupt pointer reserved by Intel for future purpose and rest of 224 interrupt pointer are available for user to use
- So when 8086 responds to a particular vector table, it goes address on the table to get the starting address of the interrupt service routine

Type no. $\times 4$ = location of IP

Hardware and Software Interrupt

- Hardware Interrupt
 - Generated by device external to processor by using pins NMI and INTR
- Software Interrupt
 - They are basically call to some predefined functions like INT instructions or divide operation that cause overflow, execution in single step etc.
 - To access BIOS and DOS procedure
 - INT 8-bit (00H to FFH)

Priority Interrupt Controllers (PIC)

- To handle interrupt from multiple devices

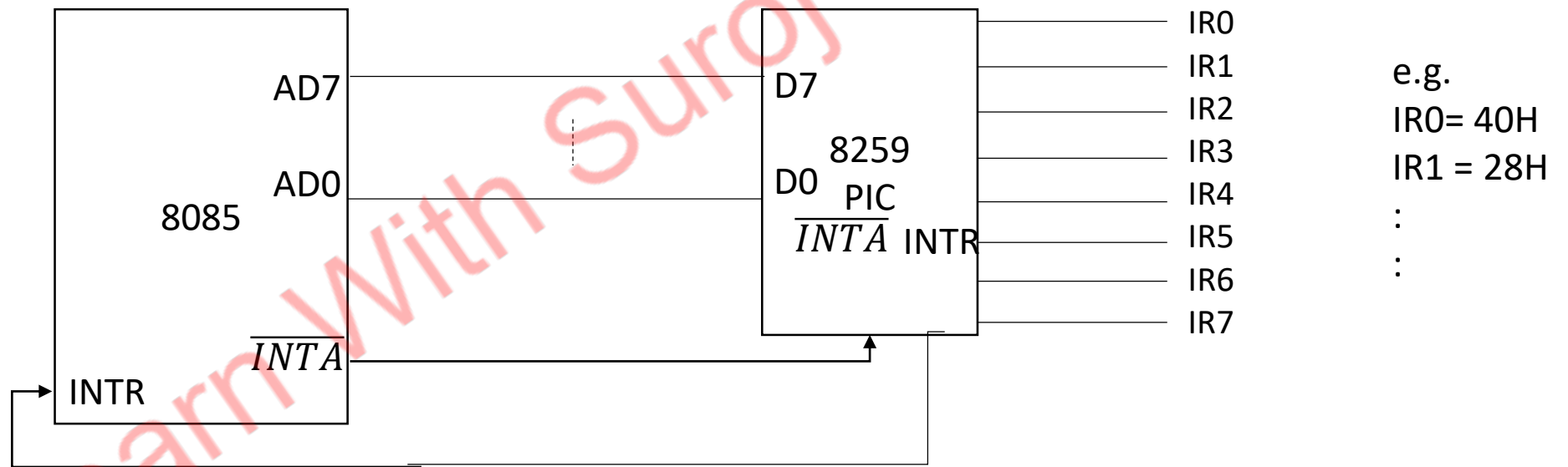


Fig: Multiple Interrupt with 8085 using PIC

Priority Interrupt Controllers (PIC)

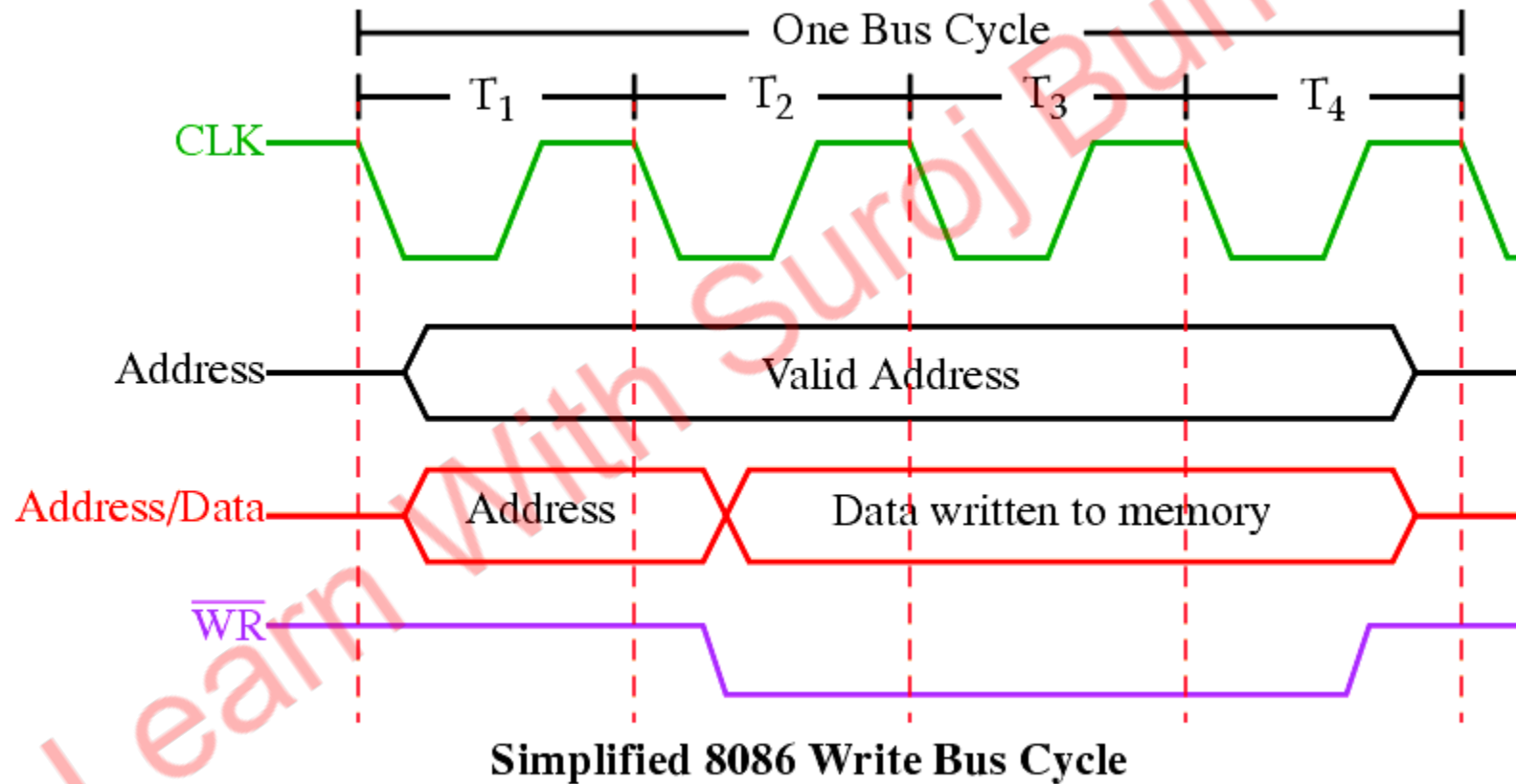
- An interrupt signal is applied to any of these interrupt cause 8059 to assert its INTR pin HIGH
- 8086 then sends two interrupt acknowledge pulses on $\overline{INTA}$ pin of PIC, the first tells the PIC to send the interrupt type to 8086 through data bus
- On receiving second $\overline{INTA}$ from 8086 the PIC outputs an interrupt types to the data bus
- The interrupt type is decided by IR input that receives interrupt signal where each has different vectors
- After getting vector type, 8086 fetches the ISR from ITV and handle the interrupt
- 8086 is capable of accepting cascading PIC, so more devices can be interfaced in interrupt mode

Priority in 8086 Interrupts

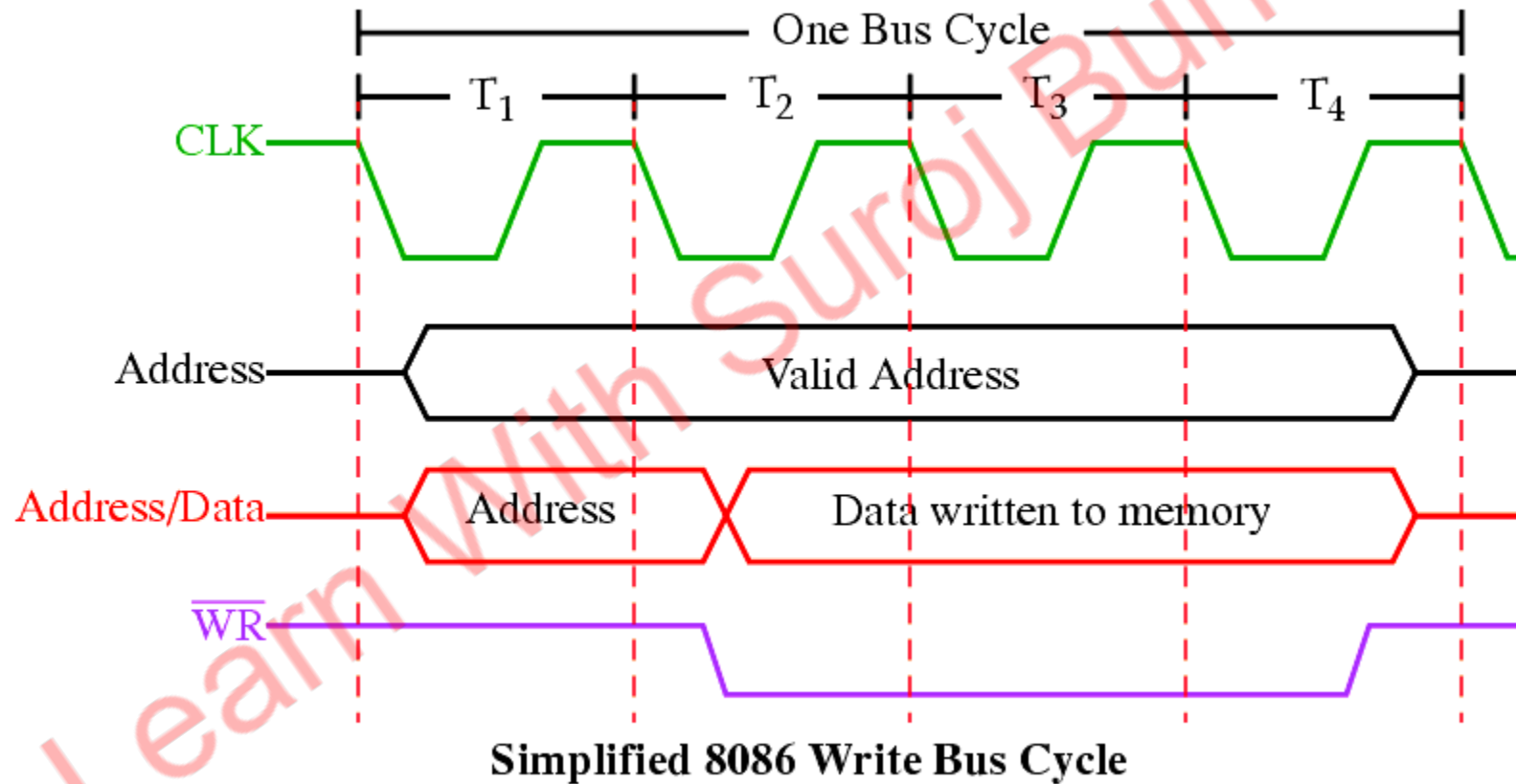
Divide Error Highest
INT n
NMI
INTR Lowest
Single step



Timing Diagram for 8086



Timing Diagram for 8086

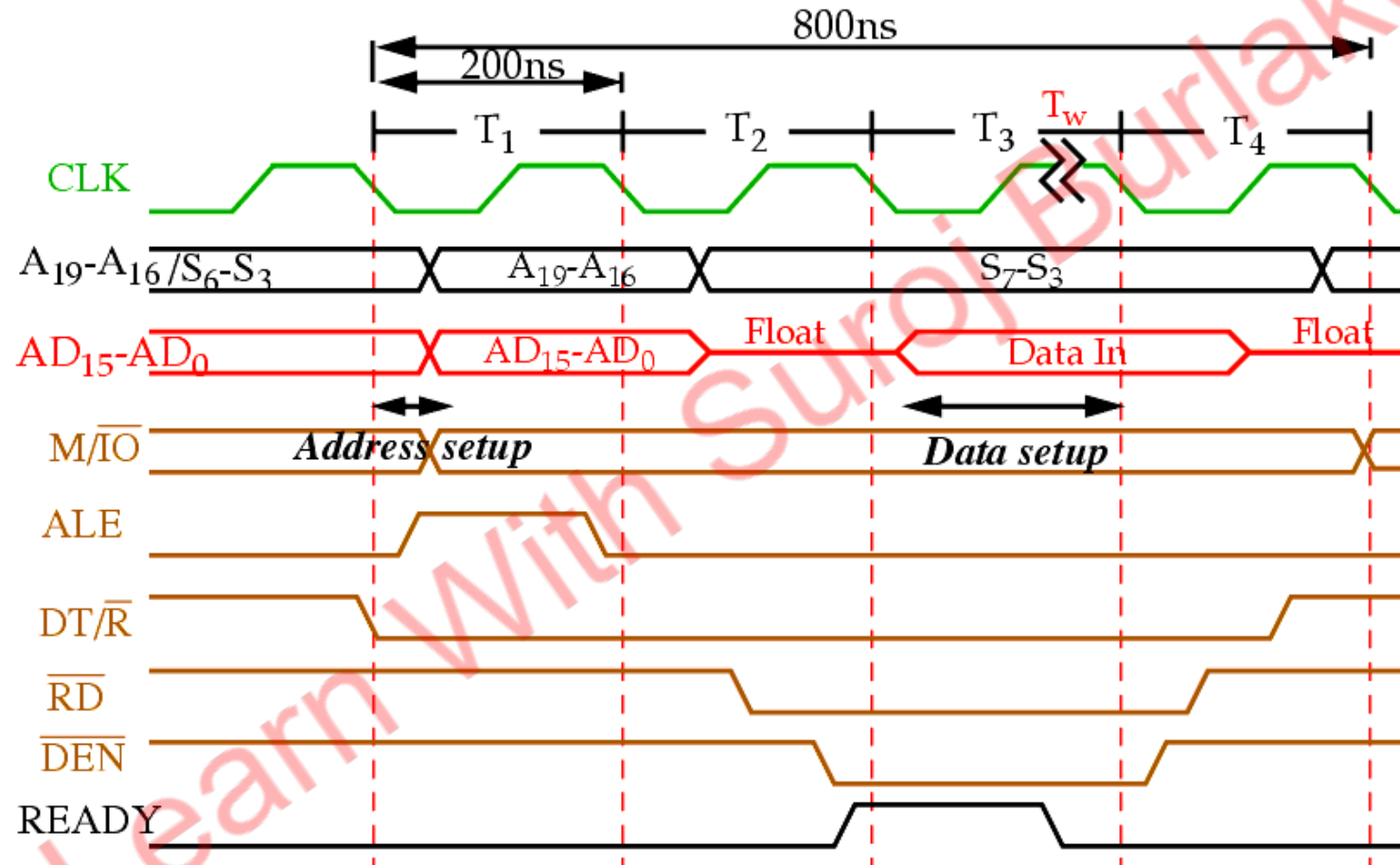


Timing Diagram for 8086

$\overline{S2}$	$\overline{S1}$	$\overline{S0}$	Function	$\overline{S2}$	$\overline{S1}$	$\overline{S0}$	Function
0	0	0	Interrupt Ack	1	0	0	Opcode Fetch
0	0	1	I/O Read	1	0	1	Memory Read
0	1	0	I/O Write	1	1	0	Memory Write
0	1	1	Halt	1	1	1	Passive

S4	S3	Function
0	0	Extra segment
0	1	Stack segment
1	0	Code or no segment
1	1	Data segment

Timing Diagram for 8086



Bus Timing for a Read Operation

Thank You

Learn With Sujoy Burlakoti